Rapport — Mini Web Application Firewall (WAF) en C

Projet universitaire — Réseaux Avancés
Date de rendu : 8 mai 2026
Auteur : Baptiste Sibellas
Langage : C (C99/C11), POSIX sockets, pthreads

---

1. Introduction

1.1 Qu'est-ce qu'un WAF ?

Un **Web Application Firewall (WAF)** est un dispositif de sécurité qui filtre et surveille le trafic HTTP entre un client et un serveur web. Contrairement à un pare-feu réseau classique qui travaille au niveau des couches 3 et 4 du modèle OSI (adresses IP, ports), le WAF opère au niveau applicatif (couche 7). Il est capable d'inspecter le contenu des requêtes HTTP — URL, paramètres, corps de requête — et de bloquer les attaques applicatives avant qu'elles n'atteignent le serveur.

Les menaces que cible un WAF sont celles référencées dans l'**OWASP Top 10**, notamment :

- **Injection SQL (SQLi)** : insertion de code SQL malveillant dans des paramètres d'entrée pour manipuler la base de données.
- **Cross-Site Scripting (XSS)** : injection de scripts JavaScript dans des pages web pour détourner les sessions utilisateur.
- **Path Traversal** : utilisation de séquences `../` pour accéder à des fichiers système en dehors de la racine web.
- **Injection de commandes** : exécution de commandes système via des paramètres non filtrés.

1.2 Objectifs du projet

Ce projet consiste à implémenter un **mini-WAF sous forme de proxy TCP en C pur**. L'objectif pédagogique est de comprendre et de mettre en pratique :

- La programmation réseau avec les sockets POSIX (TCP).
- Le multiplexage d'entrées/sorties avec `select()`.
- La gestion de la concurrence avec `pthread`.
- L'inspection de contenu HTTP au niveau applicatif.
- La journalisation thread-safe.

1.3 Technologies utilisées

Technologie	Rôle
C99/C11	Langage d'implémentation
POSIX sockets (`socket`, `bind`, `listen`, `accept`, `connect`)	Communication TCP
`select()`	Multiplexage I/O dans le relay et le logserver

Technologie	Rôle
`pthread` (POSIX Threads)	Concurrence — un thread par connexion client et par listener
`pthread_mutex_t`	Protection des écritures concurrentes dans le log et la liste SSE
`strcasestr()`	Recherche de motifs insensible à la casse
`time()` / `strftime()`	Horodatage des entrées de log
Server-Sent Events (SSE)	Streaming temps réel des logs vers le navigateur

---

2. Architecture du mini-WAF

2.1 Topologie réseau

```
[Client HTTP]
      |
      | TCP :8080
      v
[WAF — Proxy TCP]
      |          \
      | TCP :8888 \ TCP :9091 (entrées de log)
      v            v
[Serveur Web]  [LogServer]──▶ waf.log
                    |
                    | HTTP :9090 (SSE)
                    v
              [Navigateur — logs en direct]
```

Le WAF est positionné **en coupure** : le client ne connaît que le WAF (port 8080) et ignore l'existence du serveur backend (port 8888). Le WAF joue donc le double rôle de **serveur** (vis-à-vis du client) et de **client** (vis-à-vis du backend et du logserver).

2.2 Diagramme de composants

```
+-------------------+     +-------------------+     +-------------------+
|    src/client.c   |     |    src/waf.c      |     |   src/server.c    |
|                   |     |                   |     |                   |
| - forge requête   |---->| - accept()        |---->| - HTTP 200 OK     |
| - affiche réponse |     | - handle_client() |     | - corps HTML fixe |
| - retourne 0/1/2  |     | - relay()         |     |                   |
+-------------------+     +--------+----------+     +-------------------+
                                   |
                     +-------------+-------------+
                     |                           |
             +------+------+             +-------+-------+
             | src/filter.c|             | src/logger.c  |
```

```
              |             |             |                |
              | filter_simple()           | logger_init()  |
              | filter_advanced()         | logger_log() --+---> TCP :9091
              | should_block()            | logger_close() |
              +-------------+             +----------------+
                     |                           |
              +------+------+             +--------+-------+
              | rules/      |             |src/logserver.c |
              | *.txt       |             |                |
              +-------------+             | log_listener() | <-- WAF (TCP
:9091)
                                          | http_listener()| <-- Navigateur
(:9090)
                                          | broadcast_sse()|
                                          | write_log()    |---> waf.log
                                          +----------------+
```

2.3 Diagramme de séquence — Requête légitime

```
Client          WAF                 Serveur         LogServer
Navigateur
  |              |                   |               |                |
  |--TCP:8080--->|                   |               |                |
  |--GET /------>|                   |               |                |
  |              |--filter() = 0     |               |                |
  |              |--TCP:8888--------->|               |                |
  |              |--GET /------------>|               |                |
  |              |<--200 OK-----------|               |                |
  |<--200 OK-----|                   |               |                |
  |              |--logger_log()---TCP:9091---------->|               |
  |              |                   |       write_log(waf.log)        |
  |              |                   |       broadcast_sse()--SSE:9090>|
```

2.4 Diagramme de séquence — Requête bloquée

```
Client          WAF                             LogServer        Navigateur
  |              |                                 |                |
  |--TCP:8080--->|                                 |                |
  |--GET /?q=<script>->|                           |                |
  |              |--filter_simple() = 1            |                |
  |<--403 Forbidden----|                           |                |
  |              |--logger_log()---TCP:9091-------->|               |
  |              |                         write_log(waf.log)       |
  |              |                         broadcast_sse()--SSE---->|
```

2.5 Cycle de vie d'une connexion

```
accept()
   |
   +---> pthread_create(handle_client)
               |
               +---> recv() (lecture requête jusqu'à \r\n\r\n)
               |
               +---> extract first_line
               |
               +---> should_block() ?
               |        |
               |     YES: send 403 + logger_log(BLOCKED) + close(client_fd)
               |        |
               |      NO: connect(server) + send(request) + relay() +
logger_log(ALLOWED)
               |              |
               |              +---> select() bidirectionnel jusqu'à EOF
               |              +---> close(client_fd) + close(server_fd)
               |
               +---> thread exit (pthread_detach)
```

---

3. Implémentation

3.1 Gestion de la concurrence — pthreads vs select()

Choix retenu : pthreads pour les connexions, select() pour le relay.

À chaque `accept()`, un thread est créé via `pthread_create()` et immédiatement détaché avec `pthread_detach()`. Le détachement signifie que le thread libère automatiquement ses ressources à la fin de son exécution, sans qu'aucun thread parent n'ait besoin de faire `pthread_join()`. Cela évite les fuites mémoire dans une boucle infinie d'acceptation.

Ce choix est justifié par la lisibilité : chaque connexion est traitée de manière isolée dans sa propre pile d'exécution, sans état partagé (hormis le logger protégé par mutex).

Un design alternatif aurait utilisé `select()` ou `epoll()` pour tout gérer dans un seul thread (modèle event-driven). Cela serait plus performant à grande échelle, mais plus complexe à déboguer.

3.2 Moteur de filtrage

Le filtrage est séparé en deux couches :

`filter_simple()` — 8 patterns codés en dur, insensibles à la casse via `strcasestr()` (extension GNU) :

Pattern	Attaque
`<script`	XSS
`../`	Path traversal
`OR 1=1`	SQLi

Pattern	Attaque
`DROP TABLE`	SQLi
`UNION SELECT`	SQLi
`alert(`	XSS
`javascript:`	XSS
`exec(`	Command injection

`filter_advanced()` — Chargement paresseux (`lazy loading`) des fichiers `rules/simple_patterns.txt` et `rules/blocked_urls.txt` au premier appel. Chaque ligne non-vide et non-commentaire (ne commençant pas par `#`) est stockée dans un tableau statique de 256 entrées maximum. La recherche est également insensible à la casse.

`should_block()` — Appelle les deux filtres avec court-circuit : si `filter_simple` retourne 1, `filter_advanced` n'est pas évalué.

3.3 Relay bidirectionnel

La fonction `relay(int client_fd, int server_fd)` transfère les données dans les deux sens entre le client et le serveur backend, jusqu'à ce qu'une des connexions soit fermée.

`select()` est utilisé ici car il permet de surveiller simultanément les deux file descriptors sans bloquer sur l'un d'eux. Sans `select()`, un `recv()` bloquant sur `server_fd` empêcherait de traiter des données arrivant de `client_fd`, et inversement.

3.4 Journalisation centralisée — logserver

L'architecture de journalisation a été repensée pour séparer la **production** des entrées de log (côté WAF) de leur **persistance** et **diffusion** (côté logserver).

Côté WAF (`src/logger.c`) — `logger_log()` formate la ligne horodatée puis l'envoie via une connexion TCP persistante vers le logserver (port 9091). Un `pthread_mutex_t` protège le socket partagé entre les threads du WAF. En cas de déconnexion, une reconnexion automatique est tentée avant chaque envoi.

Côté logserver (`src/logserver.c`) — deux listeners tournent en parallèle dans des threads distincts :

- `log_listener()` (port 9091) : accepte les connexions du WAF, lit les lignes reçues octet par octet pour reconstituer des entrées complètes, appelle `write_log()` puis `broadcast_sse()`.
- `http_listener()` (port 9090) : sert `GET /` (page HTML) et `GET /events` (flux SSE). Les clients SSE sont stockés dans un tableau global `sse_fds[]` protégé par mutex.

Persistance : `write_log()` ouvre `waf.log` en mode **append** (`fopen("a")`). Le fichier n'est jamais réécrit entre deux lancements du logserver ; les entrées s'accumulent indéfiniment.

Interface web temps réel : quand un navigateur ouvre `http://localhost:9090/events`, le logserver envoie d'abord l'historique complet du fichier (relecture de `waf.log` ligne par ligne), puis maintient la connexion ouverte. Chaque nouvel appel à `broadcast_sse()` pousse une trame SSE (`data: <ligne>\n\n`) à tous les clients connectés. Les entrées BLOCKED s'affichent en rouge, ALLOWED en vert.

Format d'une entrée :

```
[YYYY-MM-DD HH:MM:SS] ALLOWED | 127.0.0.1 | GET / HTTP/1.0
```

---

4. Documentation utilisateur et règles de filtrage

4.1 Compilation et lancement

```
# Compilation (sans warning)
make

# Démarrer dans cet ordre (logserver doit être prêt avant le WAF)
./logserver &   # Ports 9091 (log) et 9090 (web)
./server &      # Port 8888
./waf &         # Port 8080, proxy vers 8888
sleep 1

# Observer les logs en direct (optionnel)
# Ouvrir http://localhost:9090 dans un navigateur

# Lancer les tests
bash tests/test_legitimate.sh
bash tests/test_attacks.sh
bash tests/test_concurrent.sh

# Consulter le log brut
cat waf.log
```

4.2 Fichier waf.log

Le fichier est créé dans le **répertoire courant** au lancement du **logserver** (mode append). Il n'est jamais réécrit entre deux sessions : les entrées s'accumulent. C'est le logserver — et lui seul — qui écrit dans ce fichier, évitant tout accès concurrent non contrôlé entre processus.

Chaque ligne suit le format :

```
[YYYY-MM-DD HH:MM:SS] ALLOWED|BLOCKED | <IP client> | <première ligne HTTP>
```

4.3 Tableau complet des règles actives

Motif	Type d'attaque	Source
`<script`	XSS	filter_simple (codé en dur)
`../`	Path traversal	filter_simple (codé en dur)

Motif	Type d'attaque	Source
`OR 1=1`	SQL Injection	filter_simple (codé en dur)
`DROP TABLE`	SQL Injection	filter_simple (codé en dur)
`UNION SELECT`	SQL Injection	filter_simple (codé en dur)
`alert(`	XSS	filter_simple (codé en dur)
`javascript:`	XSS	filter_simple (codé en dur)
`exec(`	Command injection	filter_simple (codé en dur)
`passwd`	Accès fichier sensible	rules/simple_patterns.txt
`/etc/shadow`	Accès fichier sensible	rules/simple_patterns.txt
`base64_decode`	Obfuscation de payload	rules/simple_patterns.txt
`eval(`	Injection de code	rules/simple_patterns.txt
`/admin`	Accès non autorisé	rules/blocked_urls.txt
`/phpmyadmin`	Accès non autorisé	rules/blocked_urls.txt
`/.env`	Exposition de configuration	rules/blocked_urls.txt
`/wp-admin`	Accès non autorisé	rules/blocked_urls.txt

4.4 Ajouter une nouvelle règle

Il suffit d'ajouter une ligne dans `rules/simple_patterns.txt` ou `rules/blocked_urls.txt`. Les commentaires commencent par `#`. Le WAF relit les fichiers au prochain démarrage (le chargement est effectué une seule fois par processus).

Exemple — bloquer les tentatives d'accès à phpinfo :

```
# dans rules/blocked_urls.txt
/phpinfo.php
```

---

5. Tests et résultats

5.1 Tableau récapitulatif

#	Requête	Résultat attendu	Résultat obtenu	Statut
1	`GET /`	200	200	PASS
2	`GET /index.html`	200	200	PASS
3	`GET /about?name=John`	200	200	PASS
4	`POST /form` (data normale)	200	200	PASS

#	Requête	Résultat attendu	Résultat obtenu	Statut
5	`GET /?q=<script>alert(1)</script>`	403	403	PASS
6	`GET /?q=alert(xss)`	403	403	PASS
7	`GET /?url=javascript:void(0)`	403	403	PASS
8	`GET /?id=1 OR 1=1`	403	403	PASS
9	`GET /?q=DROP TABLE users`	403	403	PASS
10	`GET /?q=UNION SELECT * FROM users`	403	403	PASS
11	`GET /../../etc/passwd`	403	403	PASS
12	`GET /?cmd=exec(ls)`	403	403	PASS
13	`GET /admin`	403	403	PASS
14	`GET /phpmyadmin`	403	403	PASS
15	`GET /.env`	403	403	PASS
16	`GET /wp-admin`	403	403	PASS
17	`GET /?file=passwd`	403	403	PASS
18	`GET /?q=base64_decode(xyz)`	403	403	PASS
19	`GET /?q=eval(code)`	403	403	PASS
20	5 clients `GET /` simultanés	5 × 200	5 × 200	PASS

Résultat global : 20/20 cas passés.

5.2 Capture de waf.log après exécution

```
[2026-05-07 22:05:44] ALLOWED | 127.0.0.1 | GET / HTTP/1.0
[2026-05-07 22:05:44] ALLOWED | 127.0.0.1 | GET / HTTP/1.0
[2026-05-07 22:05:44] ALLOWED | 127.0.0.1 | GET / HTTP/1.0
[2026-05-07 22:05:44] ALLOWED | 127.0.0.1 | GET / HTTP/1.0
[2026-05-07 22:05:44] ALLOWED | 127.0.0.1 | GET / HTTP/1.0
```

Les 5 entrées ALLOWED à la même seconde confirment le traitement concurrent sans deadlock.

5.3 Test de concurrence

Le test `tests/test_concurrent.sh` lance 5 processus `./client` en arrière-plan avec `&` et attend leur terminaison avec `wait`. Le WAF crée un thread par connexion entrante ; le mutex du logger garantit que les 5 écritures dans `waf.log` se font sans interférence.

5.4 Limites et pistes d'amélioration

Limites observées :

- **Faux positifs** : le pattern `passwd` bloque tout URL ou paramètre contenant cette chaîne, y compris des usages légitimes (ex. : `?action=changepassword`).
- **Taille de buffer fixe** : les requêtes dépassant 8 192 octets sont tronquées ; des payloads fragmentés sur plusieurs segments TCP pourraient échapper au filtrage.
- **Chargement des règles non rechargeable à chaud** : modifier `rules/*.txt` nécessite un redémarrage du WAF.
- **Un thread par connexion** : ne passe pas à l'échelle pour des milliers de connexions simultanées (préférer `epoll` + pool de threads).
- **Pas de TLS** : le WAF inspecte uniquement du HTTP en clair.

Pistes d'amélioration :

- Implémenter un rechargement des règles via signal SIGUSR1.
- Ajouter un mode liste blanche (allowlist) pour certains chemins.
- Journaliser le corps complet de la requête bloquée pour faciliter l'audit.
- Utiliser des expressions régulières (POSIX `regcomp`/`regexec`) pour des règles plus expressives.

---

6. Conclusion

6.1 Bilan pédagogique

Ce projet a permis de mettre en œuvre de manière concrète les concepts fondamentaux des réseaux : sockets TCP, multiplexage I/O avec `select()`, concurrence avec `pthread` et mutex. L'implémentation d'un proxy applicatif illustre la différence entre filtrage réseau et filtrage applicatif, et montre pourquoi les pare-feux classiques sont insuffisants face aux attaques de couche 7.

La modularisation du code en composants distincts (`filter`, `logger`, `waf`, `logserver`) reflète les bonnes pratiques de développement système et facilite l'évolution indépendante de chaque module. Le découplage entre la production des logs (WAF) et leur persistance/diffusion (logserver) illustre le pattern **producteur–consommateur** appliqué à la journalisation réseau. L'utilisation des Server-Sent Events pour l'interface web démontre qu'un protocole de streaming temps réel peut être implémenté en C pur sans bibliothèque externe.

6.2 Comparaison avec des WAF industriels

Critère	Mini-WAF (ce projet)	ModSecurity	Cloudflare WAF
Règles	Patterns texte simples	Langage SecRule (PCRE)	Règles managées + ML
Performance	~5 clients simultanés	Milliers de req/s	Millions de req/s
TLS	Non	Oui (via Apache/Nginx)	Oui
Mise à jour des règles	Redémarrage	Rechargement à chaud	Temps réel
Faux positifs	Élevés	Configurables	Faibles (ML)
Journalisation	Logserver TCP + fichier	Fichier local	Dashboard cloud

Critère	Mini-WAF (ce projet)	ModSecurity	Cloudflare WAF
Interface de monitoring	Web SSE (port 9090)	ModSecurity Console	Dashboard Cloudflare
Déploiement	Local uniquement	Module Apache/Nginx	Cloud CDN

ModSecurity (https://modsecurity.org) est le WAF open-source de référence, intégré à Apache et Nginx, et utilisant le langage de règles **SecRule** basé sur des expressions régulières PCRE. Cloudflare WAF (https://www.cloudflare.com/fr-fr/application-services/products/waf/) opère au niveau du CDN et intègre des mécanismes d'apprentissage automatique pour réduire les faux positifs. Notre mini-WAF est fonctionnellement analogue dans ses principes (proxy TCP + inspection de contenu), mais sans la performance ni la sophistication de ces solutions.

---

7. Références

- OWASP ModSecurity : https://modsecurity.org
- Cloudflare WAF : https://www.cloudflare.com/fr-fr/application-services/products/waf/
- MDN — Server-Sent Events : https://developer.mozilla.org/fr/docs/Web/API/Server-sent_events
- Man pages :
  - `socket(7)` — API sockets POSIX
  - `pthread_create(3)` — création de threads
  - `select(2)` — multiplexage I/O
  - `strcasestr(3)` — recherche insensible à la casse
  - `strftime(3)` — formatage de dates
  - `send(2)` — `MSG_NOSIGNAL` pour éviter SIGPIPE sur socket fermé